

Project Question

Aircraft engine failures cause costly maintenance and operational disruption. Predictive maintenance estimates an engine's **Remaining Useful Life (RUL)**, the number of operating cycles before failure, so maintenance can be scheduled proactively.
Can RUL be predicted from sensor measurements and operational settings, and which model performs best?

We compare three regression models, and test a second question: does adding engine age (cycle number) as a feature help or hurt prediction?

Dataset

NASA C-MAPSS turbofan engine simulation data (cleaned Kaggle release). Each engine is a multivariate time series of 3 operational settings and 21 sensor measurements, with a computed RUL at every operating cycle.

100 / 260

engines (FD001 / FD004)

21

sensor channels

Two scenarios are studied: **FD001** (one operating condition, one fault mode) and **FD004** (six operating conditions and multiple fault modes—the hardest benchmark in the suite).

Preprocessing: we cap the training RUL at 125 cycles (healthy engines carry little degradation signal), drop sensors that never vary, and standardize all features.

Cross-Validation & Model Selection

- Hyperparameters (regularization α and network architecture) are tuned by 5-fold cross-validation.
- Grouped split by engine: all cycles of one engine stay within a single fold.
- This prevents leakage. Consecutive cycles of an engine are nearly identical, so a random split would let the model memorize neighbors and report falsely low error.
- For FD004, rows are clustered into the six operating regimes and each sensor is standardized within its regime, removing operating-condition effects so the degradation signal is visible.

Raw Engine Data

21 sensors + 3 operational settings

Preprocess

cap RUL at 125, drop dead sensors, standardize

Train + 5-Fold CV

grouped by engine; tune hyperparameters

Predict RUL

Ridge / Lasso / Neural Network

Figure. Analysis pipeline, from raw sensor data to predicted RUL.

Machine Learning Methods

Three regression models predict a continuous RUL value. Ridge and Lasso are linear baselines; the neural network introduces nonlinearity.

Ridge Regression

A linear model with an L2 penalty that shrinks all coefficients toward zero to control overfitting. Fast, stable, and a natural baseline.

Lasso Regression

A linear model with an L1 penalty. Beyond regularizing, it performs feature selection by driving unhelpful coefficients exactly to zero, revealing which sensors matter.

Neural Network

A nonlinear multilayer perceptron with ReLU activations, dropout, and early stopping. It represents the curved, accelerating sensor-to-RUL relationships that linear models cannot capture.

Key Findings

- The neural network achieves the lowest error on every metric and both scenarios.
- Ridge and Lasso perform almost identically, so the predictive signal is spread across many sensors rather than concentrated in a few.
- Lasso drives the operational settings to zero in FD001—correctly recovering that they barely vary under a single operating condition.
- Strongest sensors by Lasso weight: 9, 11, 12, 4, 7, and 14.

Most influential sensors (FD001)

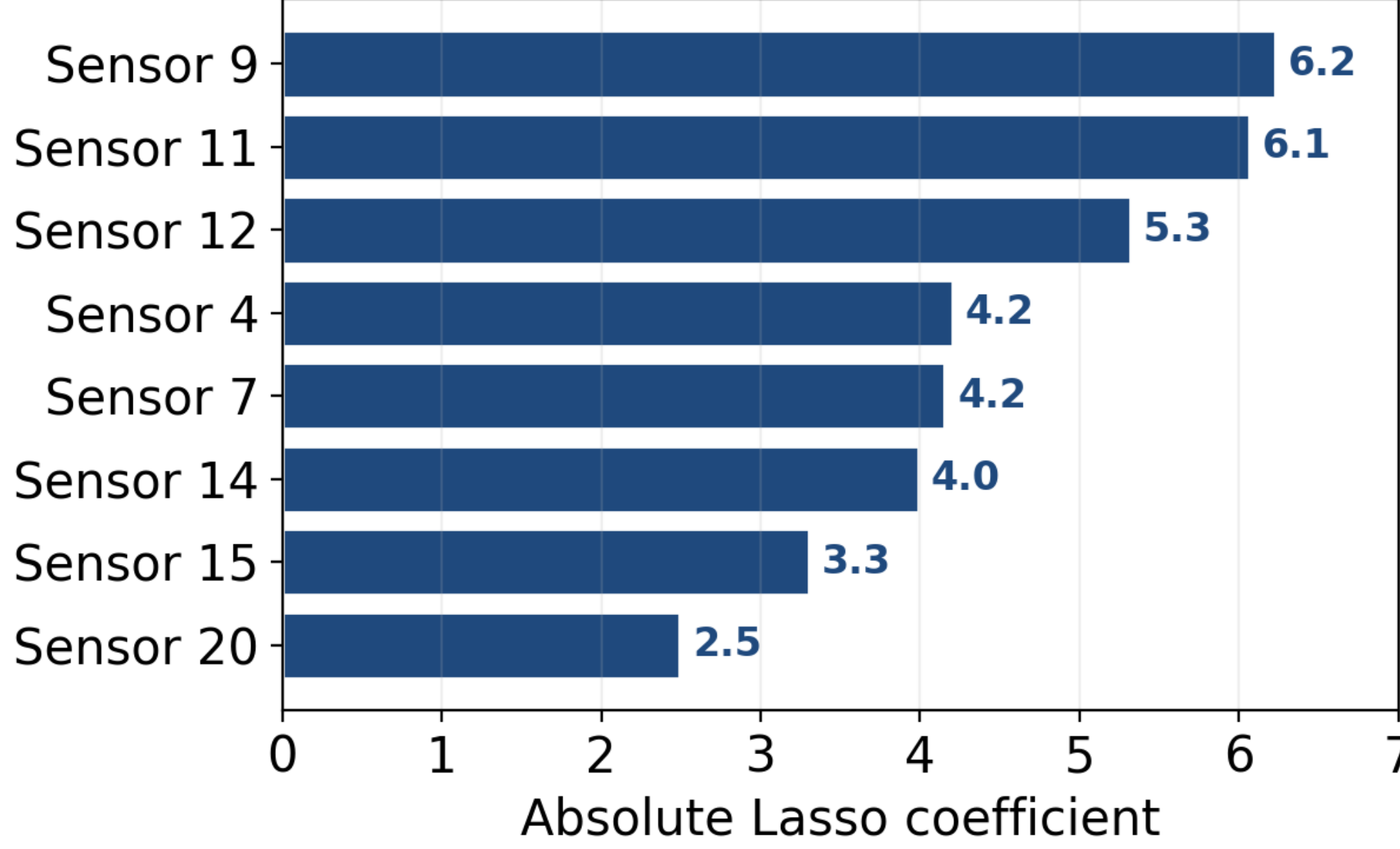


Figure. Lasso coefficient magnitudes: predictive weight spreads across many sensors; operational settings go to zero.

Takeaway

Because no single sensor dominates, the linear models (Ridge and Lasso) land in nearly the same place. Therefore, regularization choice matters little here. The neural network's edge comes from combining many weak signals nonlinearly, not from weighting any one sensor differently.

Model Comparison Results

On FD001, the neural network leads on every metric, improving RMSE by around 3.6 cycles and explaining 81% of the variance versus 73% for the linear models.

Model (FD001)	RMSE	MAE	R ²
Ridge	21.67	17.44	0.728
Lasso	21.73	17.48	0.727
Neural Net	18.10	13.11	0.810

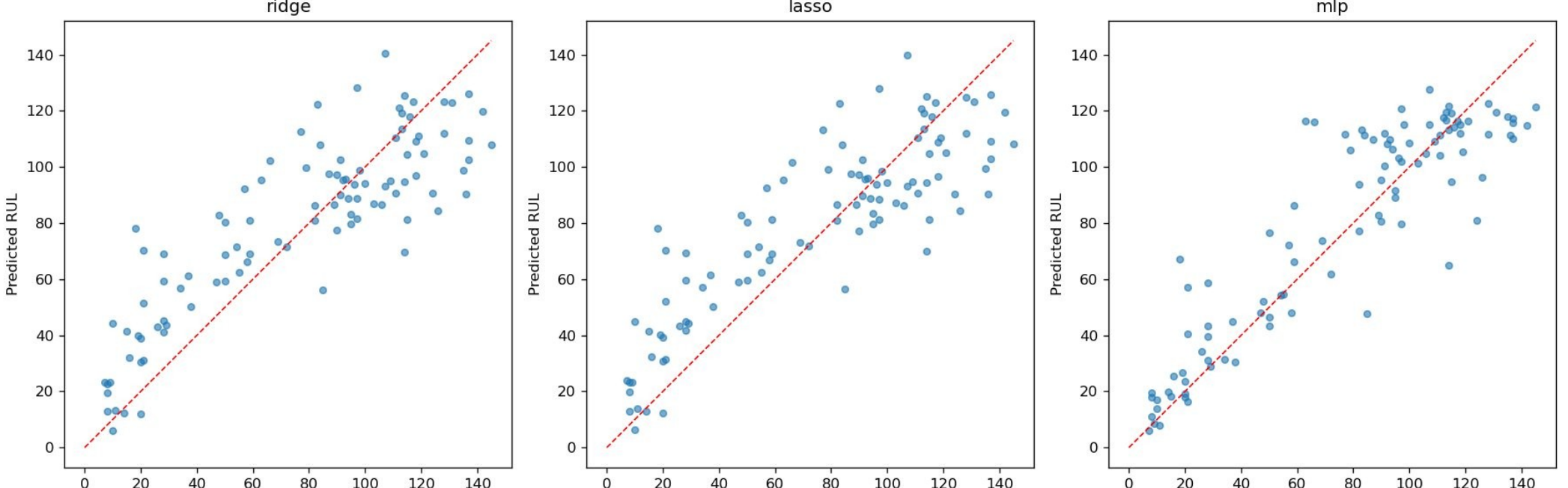


Figure. Predicted vs. true RUL (FD001). The neural network clusters tightest on the diagonal.

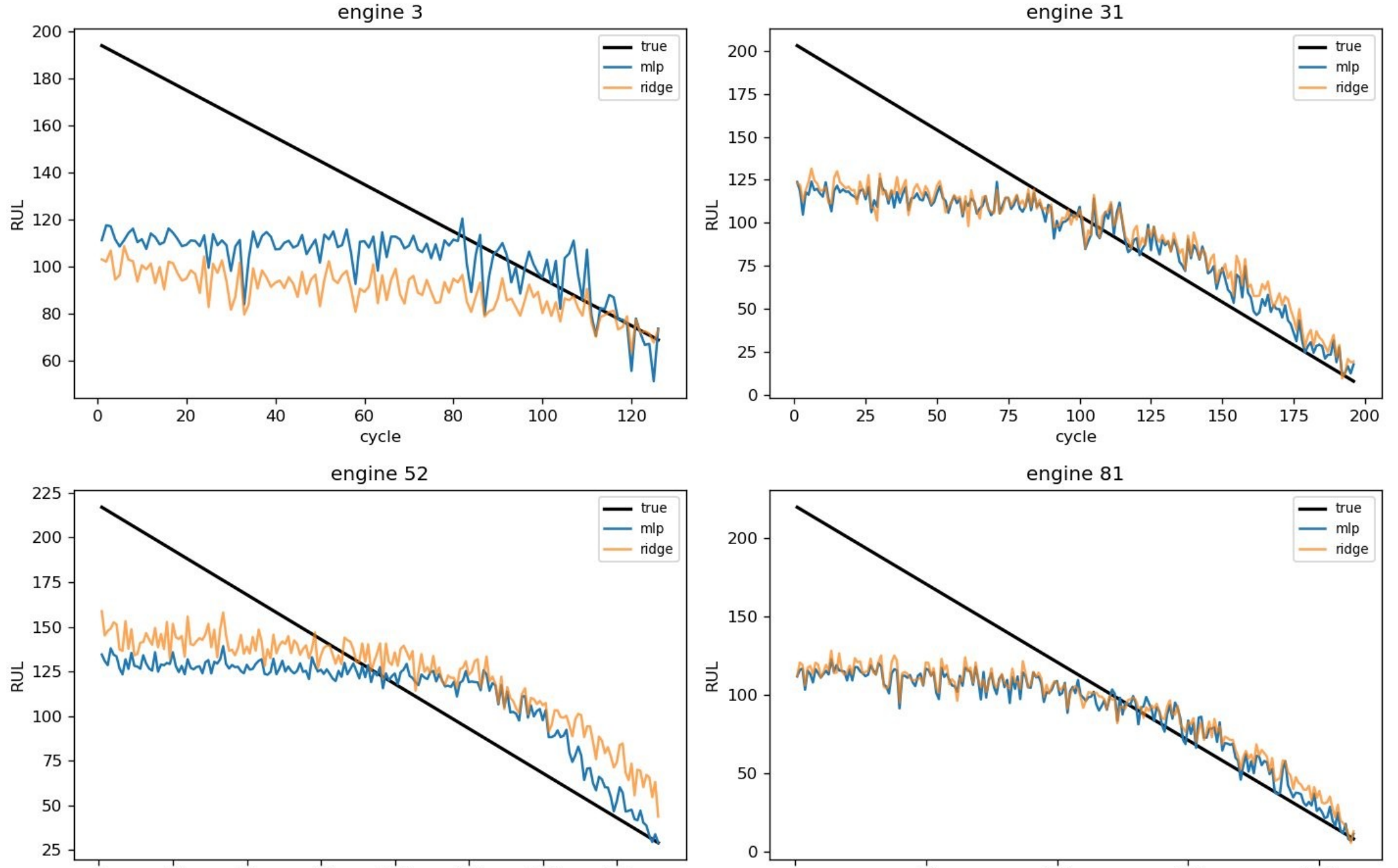


Figure. Per-engine RUL trajectories. Predictions converge on the true line as failure nears.

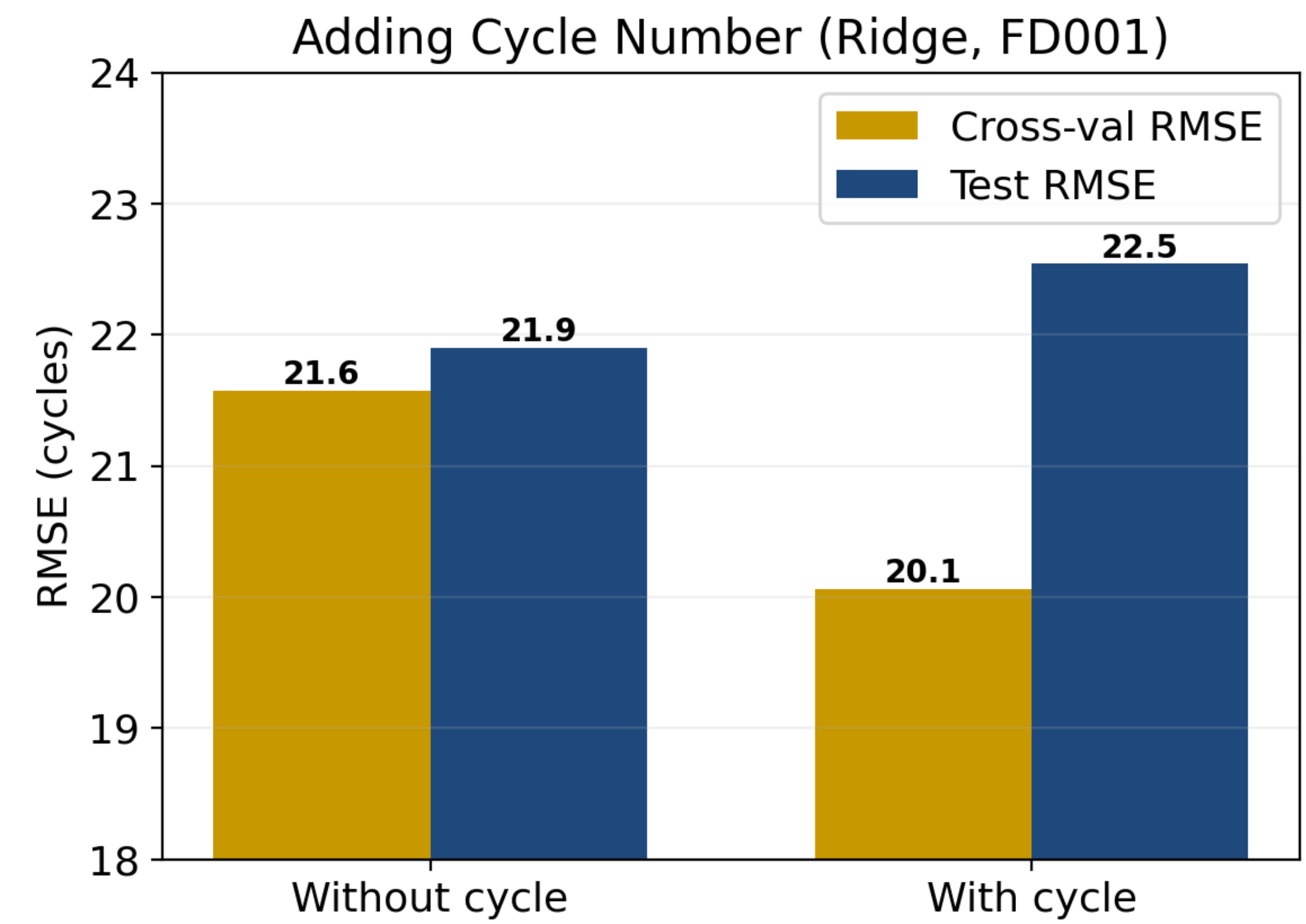


Figure. Adding cycle number lowers cross-validation error but raises test error—a misleading feature.

Scaling to the Hard Case (FD004)

On FD004, with six operating conditions and multiple fault modes, every model's error roughly doubles. The neural network's advantage over the linear models **grows** (Δ RMSE rises from 3.6 to 4.5 cycles), showing nonlinearity matters more as the problem gets harder.

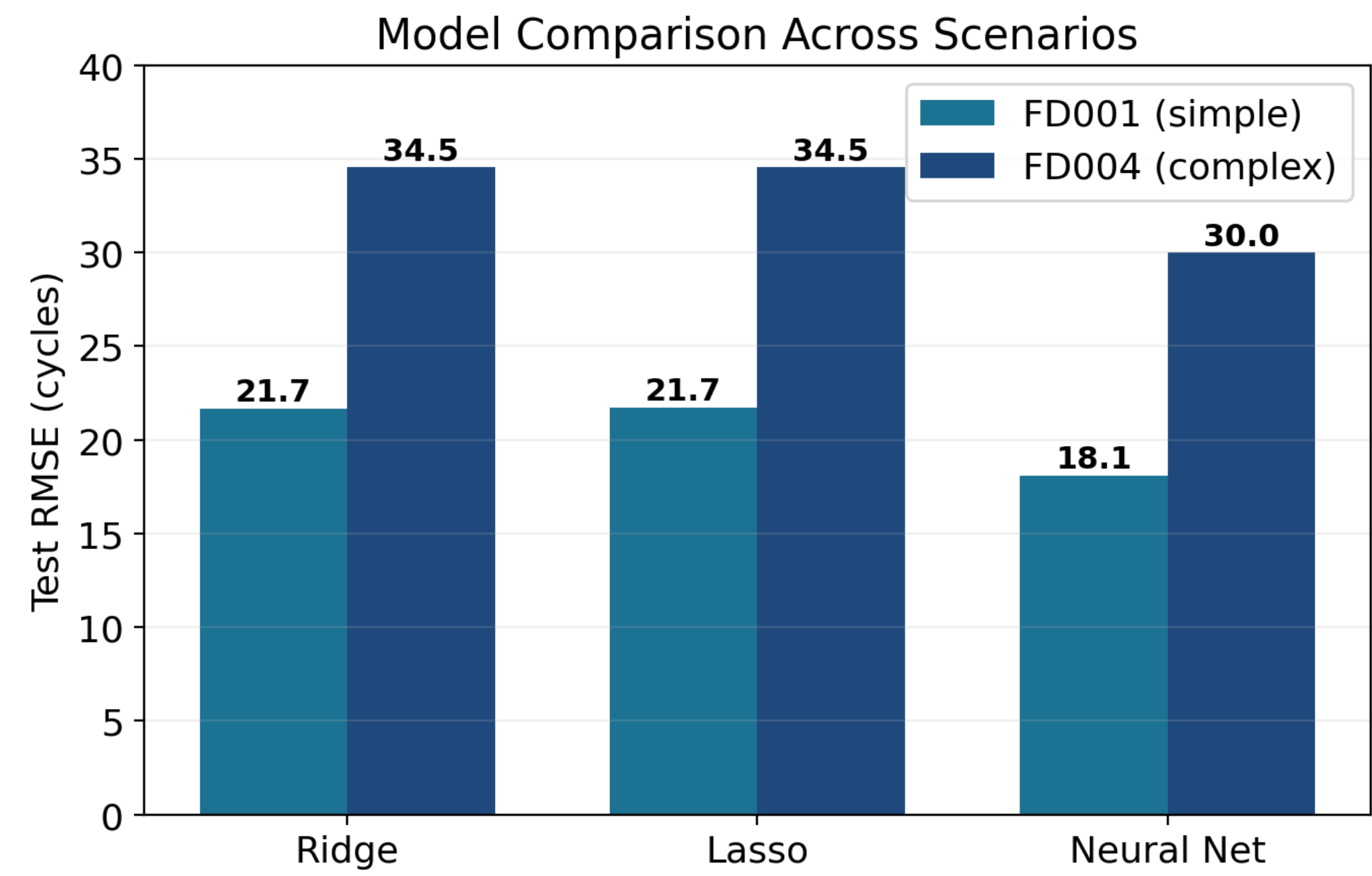


Figure. Test RMSE across models and both scenarios.

FD004: Neural Network Predictions

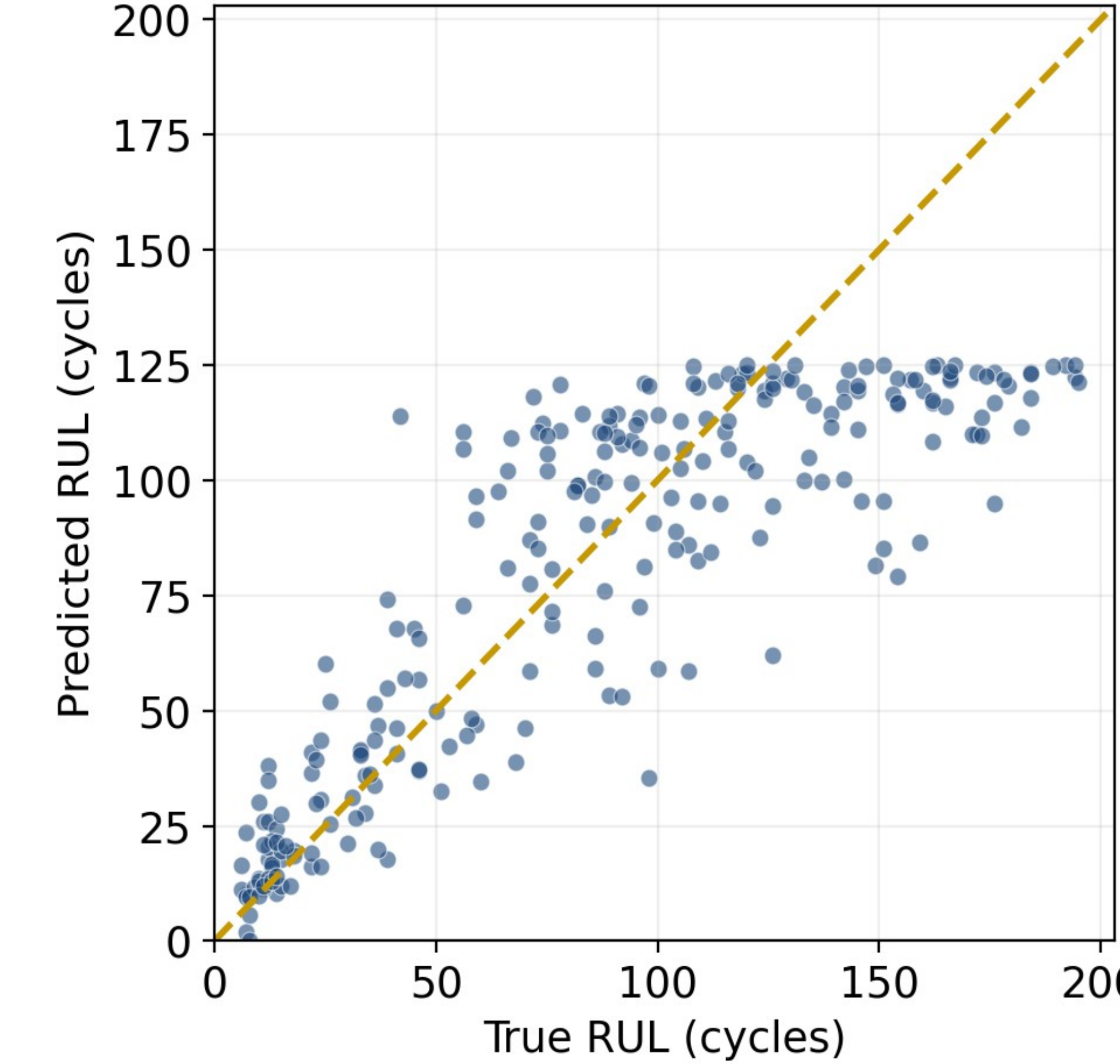


Figure. Neural-network predictions on FD004 test engines.

Conclusions & Future Work

- RUL is predictable from sensors and settings with good accuracy (R^2 up to 0.81).
- Neural networks win on both scenarios, and their edge widens as the problem grows more complex and nonlinear.
- Ridge and Lasso are nearly tied, indicating signal spread across many sensors.
- Cycle number hurts test accuracy: it helps in training but fails on truncated test trajectories.
- Future work: engineer time-derivative (rate-of-change) sensor features to capture how fast an engine trends toward failure.